OPERATING SYSTEMS

UNIT – 1: OPERATING SYSTEMS OVERVIEW & SYSTEM STRUCTURES

SECTION B – 5-MARK LONG ANSWER QUESTIONS

All 16 questions | Expanded into the approved 10-mark answer format

Answer format: easy English, clear explanations, examples, labelled diagrams, comparison tables and conclusions.

UNIT – 1: OPERATING SYSTEMS OVERVIEW & SYSTEM STRUCTURES

SECTION B – 5-MARK LONG ANSWER QUESTIONS

Each answer is expanded into the approved 10-mark format

Q1. Explain in detail the functions of an Operating System.

5 Marks – 10 Marks Answer Format

1. Introduction

An Operating System (OS) is system software that acts as a bridge between the user and computer hardware.

It helps the user to operate the computer easily. It also controls and manages all the resources of a computer.

Examples: Windows, Linux, Android and macOS.

2. Main Functions of an Operating System

An Operating System performs the following important functions:

1. Process Management

- A process is a program that is currently running.
- The OS manages all running programs.
- It decides which program gets the CPU first.
- It also creates and deletes processes.

Example: When we open Chrome and a music player at the same time, the OS manages both programs.

2. Memory Management

- Memory management means managing the computer's RAM.
- The OS gives memory to programs when they need it.
- It takes back memory when programs are closed.
- It prevents programs from using memory allocated to other programs.

Example: When we open a game, the OS provides RAM to run that game.

3. File Management

- The OS manages files and folders in a computer.
- It helps us to create, delete, open and save files.
- It also organizes files into folders.
- It controls access to files.

Example: Creating a folder and saving photos inside it.

4. Device Management

- The OS manages hardware devices connected to the computer.
- It controls devices using special software called device drivers.
- It manages devices like printers, keyboards, mouse and speakers.

Example: When we connect a printer, the OS helps the computer communicate with it.

5. Security and Protection

- The OS protects computer data from unauthorized users.
- It provides passwords and user accounts.

- It controls who can access files and programs.
- It protects system resources.

Example: A password is required to log in to a computer.

6. User Interface

- A user interface helps users communicate with the computer.
- It allows users to give commands and perform tasks.
- The OS provides different types of interfaces.

Types:

- CLI – Command Line Interface.
- GUI – Graphical User Interface.
- Touch Interface.

Example: Clicking icons in Windows is an example of GUI.

7. Secondary Storage Management

- The OS manages storage devices like hard disks and SSDs.
- It keeps track of available storage space.
- It decides where files are stored.
- It helps users read and write data.

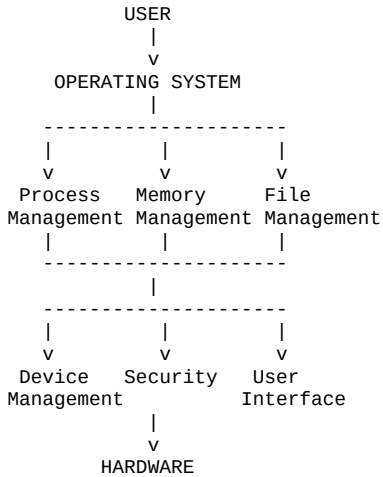
Example: Saving a movie on a computer's hard disk.

8. Error Detection and Handling

- The OS checks the computer for errors.
- It detects problems in hardware and software.
- It takes suitable action to handle errors.
- It helps the computer work properly.

Example: When a printer is not connected, the OS displays an error message.

3. Diagram – Functions of Operating System



4. Advantages of Operating System

1. Makes the computer easy to use.
2. Manages all computer resources.
3. Provides security to data.
4. Allows multiple programs to run.

5. Helps hardware and software work together.

5. Conclusion

An Operating System is an important part of a computer. It manages processes, memory, files, devices and security. Without an OS, using a computer would be very difficult for normal users.

Remember: The main work of an OS is to manage computer resources and make the computer easy to use.

Important Points to Remember for Exams

Function	Easy meaning
Process Management	Manages running programs
Memory Management	Manages RAM
File Management	Manages files and folders
Device Management	Manages hardware devices
Security	Protects data
User Interface	Helps users interact
Storage Management	Manages disk space
Error Handling	Finds and handles errors

Q2. Describe the types of system calls.

5 Marks – 10 Marks Format

1. Introduction

A system call is a way through which a user program requests a service from the Operating System.

It acts as a bridge between an application program and the Operating System.

Example: When we save a file, the application uses a system call to ask the OS to save the data.

2. Types of System Calls

There are five main types of system calls.

1. Process Control

- Process control system calls manage running programs.
- They help to create and terminate processes.
- They also help to stop and restart processes.
- They allow one process to wait for another process.

Example: Opening a calculator creates a new process.

Common system calls: `fork()`, `exec()`, `exit()`, `wait()`.

2. File Management

- These system calls manage files and folders.
- They help to create and delete files.
- They allow programs to read and write data.
- They also help to open and close files.

Example: Creating a text file and saving information in it.

Common system calls: `open()`, `read()`, `write()`, `close()`.

3. Device Management

- These system calls manage input and output devices.

- They help programs request hardware devices.
- They control devices like printers, keyboards and disks.
- They also release devices after use.

Example: Sending a document to a printer.

4. Information Maintenance

- These system calls provide information about the computer.
- They help to get the current date and time.
- They provide information about processes and system details.
- They can also change certain system information.

Example: Checking the current system date and time.

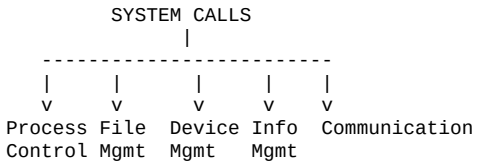
Common system calls: `getpid()`, `time()`.

5. Communication

- These system calls allow processes to communicate with each other.
- They help processes exchange data.
- They support sending and receiving messages.
- They are useful in network communication.

Example: Two programs exchanging information.

3. Diagram – Types of System Calls



4. How System Calls Work

1. The user starts an application.
2. The application requests an OS service.
3. A system call transfers the request to the OS.
4. The OS performs the required operation.
5. The result is returned to the application.

Example: When we open a file, the application requests the OS to open it. The OS finds the file and returns the result.

5. Advantages of System Calls

1. They provide communication between programs and the OS.
2. They help manage computer resources.
3. They provide controlled access to hardware.
4. They support file and process management.
5. They improve system security.

6. Conclusion

System calls are an important part of an Operating System. They allow application programs to request services such as process control, file management, device management, information maintenance and communication.

Remember: System calls act as a bridge between user programs and the Operating System.

Q3. Discuss the importance of debugging in Operating System development.

5 Marks – 10 Marks Format

1. Introduction

Debugging is the process of finding and correcting errors in a computer program.

During Operating System development, programmers may make mistakes in the code. These mistakes are called bugs.

Debugging helps programmers find and fix these bugs.

Example: If a computer hangs while opening an application, programmers use debugging to find the reason.

2. Importance of Debugging

1. Finds Errors

- Debugging helps identify errors in the OS.
- It finds problems in program instructions.
- It helps programmers understand why a program is not working.

Example: Finding an error that prevents the computer from starting.

2. Improves System Performance

- Debugging helps find slow operations.
- Programmers can correct inefficient code.
- It makes the Operating System work faster.

Example: Fixing a problem that makes a computer slow.

3. Improves Security

- Debugging helps find security-related errors.
- It identifies weaknesses in the OS.
- It helps prevent unauthorized access and attacks.

Example: Finding and correcting a bug that allows unauthorized users to access data.

4. Prevents System Crashes

- A bug can cause the computer to stop working.
- Debugging helps find the reason for system crashes.
- It helps make the OS more stable.

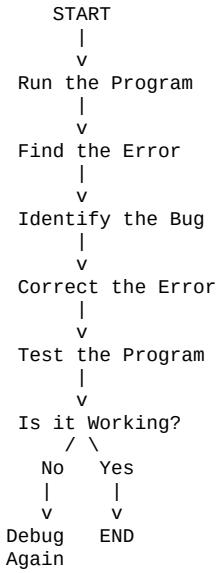
Example: Fixing an error that causes a blue screen in Windows.

5. Makes the System Reliable

- Debugging improves the quality of the OS.
- It reduces unexpected errors.
- It helps the computer work properly for a long time.

Example: Fixing errors before releasing a new OS update.

3. Debugging Process Diagram



4. Common Debugging Tools

1. **GDB:** Helps programmers find errors in programs.
2. **Print Statements:** Display values to understand program execution.
3. **Kernel Debugger:** Helps find errors inside the OS kernel.
4. **Log Files:** Store information about system activities and errors.

5. Advantages of Debugging

1. Reduces program errors.
2. Improves system performance.
3. Increases security.
4. Prevents system crashes.
5. Improves reliability.

6. Conclusion

Debugging is very important in Operating System development. It helps programmers find and correct errors, improve performance and make the OS secure and reliable.

Remember: Debugging means finding and fixing mistakes in a program.

Q4. Explain in detail the structure of an Operating System.

5 Marks – 10 Marks Format

1. Introduction

Operating System structure describes how different parts of an OS are organized and how they work together.

An Operating System contains different components such as the kernel, file system, device drivers and user interface.

Each component performs a specific task.

2. Main Components of an Operating System

1. Kernel

- The kernel is the main part of an Operating System.
- It directly communicates with computer hardware.
- It manages CPU, memory and devices.

- It controls important system operations.

Example: The kernel manages CPU time when multiple programs are running.

2. Process Management

- It manages all running programs.
- It creates and deletes processes.
- It decides which process gets CPU time.
- It supports multitasking.

Example: Running a browser and music player together.

3. Memory Management

- It manages the computer's main memory (RAM).
- It gives memory to programs when needed.
- It takes back memory after a program finishes.
- It keeps track of memory usage.

Example: Providing RAM to a game when it starts.

4. File System

- It manages files and folders.
- It allows users to create, delete and save files.
- It organizes data on storage devices.
- It controls file access.

Example: Saving a document in a folder.

5. Device Drivers

- Device drivers help the OS communicate with hardware.
- They control devices such as printers and keyboards.
- They help hardware work correctly.

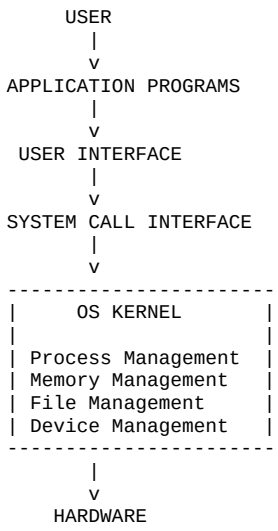
Example: A printer driver helps the OS communicate with a printer.

6. User Interface

- It allows users to communicate with the OS.
- It accepts commands from users.
- It displays information on the screen.
- It can be graphical or command-based.

Example: Windows desktop is a GUI.

3. Diagram – Basic Structure of an Operating System



4. Types of Operating System Structures

1. **Monolithic Structure:** All major OS services work inside one large kernel.
2. **Layered Structure:** The OS is divided into different layers.
3. **Microkernel Structure:** Only essential services remain inside the kernel; other services run separately.
4. **Modular Structure:** The kernel is divided into separate modules.

5. Advantages of OS Structure

1. Makes the OS organized.
2. Helps manage hardware and software.
3. Makes maintenance easier.
4. Improves system performance.
5. Helps developers find and correct errors.

6. Conclusion

Operating System structure explains how different OS components are arranged and connected. These components work together to manage the computer and provide services to users.

Remember: The kernel is the heart of the Operating System.

Q5. Explain the difference between system programs and application programs.

5 Marks – 10 Marks Format

1. Introduction

Computer programs are divided into different types based on their purpose.

Two important types are:

1. System Programs
2. Application Programs

System programs help the computer operate, while application programs help users perform specific tasks.

2. System Programs

- System programs provide a useful environment for using the computer.
- They help manage files, devices and system operations.

- They support the working of the Operating System.
- They help users and programmers perform system-related tasks.

Examples:

- File management tools.
- Command interpreters.
- System utilities.
- Compilers.

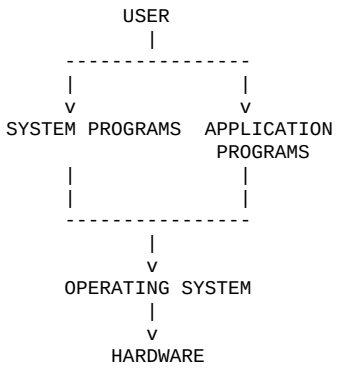
3. Application Programs

- Application programs are designed to perform specific tasks for users.
- They help users complete their daily activities.
- They run using services provided by the OS.
- They are created for different purposes.

Examples:

- MS Word – Writing documents.
- Google Chrome – Browsing the internet.
- VLC – Playing videos.
- Calculator – Performing calculations.

4. Diagram – System Programs and Application Programs



5. Differences Between System Programs and Application Programs

System Programs	Application Programs
Help in operating and managing the computer.	Help users perform specific tasks.
Support the OS environment.	Use OS services.
Mainly work with system operations.	Mainly work with user tasks.
Often come with the OS.	Can be installed separately.
Examples: Command interpreter, system utilities.	Examples: MS Word, Chrome, VLC.

6. Real-Life Example

Imagine a school:

- System programs are like the school management staff who maintain the school.
- Application programs are like the classrooms where students perform their activities.

Both are important for the school to work properly.

Similarly, system programs and application programs are important for a computer.

7. Advantages

System Programs:

1. Help manage computer resources.
2. Provide useful system tools.
3. Support program execution.

Application Programs:

1. Make user tasks easier.
2. Save time.
3. Support different activities.

8. Conclusion

System programs help manage and operate the computer environment, while application programs help users perform specific tasks. Both work with the Operating System.

Remember: System programs help the computer work; application programs help users work.

Q6. Write short notes on Open-Source Operating Systems.

5 Marks – 10 Marks Format

1. Introduction

An Open-Source Operating System is an OS whose source code is freely available for people to view, use, modify and share according to its licence.

Source code means the instructions written by programmers to create software.

Examples: Linux, Ubuntu, Fedora and Debian.

2. Features of Open-Source Operating Systems

1. Free Availability

- Open-source OS software is often available free of cost.
- Users can download and use it.
- Some distributions may provide paid support or services.

Example: Ubuntu can be downloaded and installed for free.

2. Source Code Availability

- Its source code is available to users.
- Programmers can study how the OS works.
- They can understand and learn from the code.

Example: Developers can study the Linux kernel source code.

3. Modification

- Users can modify the source code.
- They can add new features.
- They can improve existing features according to their needs.

Example: Developers can customize Linux for a particular device.

4. Security

- Developers can examine the source code.
- They can identify and correct security problems.

• Security updates can be released by the community or maintainers.

Example: Linux security bugs can be reported and fixed.

5. Community Support

• Many developers work together.

• They share knowledge and solve problems.

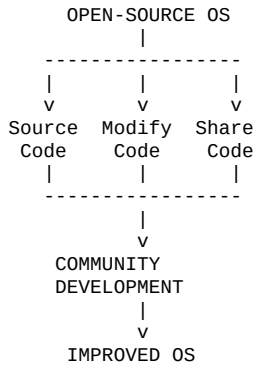
• They improve the OS regularly.

Example: Linux has a large worldwide developer community.

3. Examples of Open-Source Operating Systems

1. **Ubuntu:** A popular Linux-based OS used on computers and servers.
2. **Fedora:** A Linux-based OS used by developers and general users.
3. **Debian:** A stable Linux-based OS used on computers and servers.

4. Diagram – Open-Source Operating System



5. Advantages

1. Often available free of cost.
2. Source code can be studied.
3. Can be customized.
4. Supported by developer communities.
5. Encourages innovation and learning.

6. Disadvantages

1. Some versions may be difficult for beginners.
2. Certain hardware may need additional drivers.
3. Professional support may cost money.
4. Some commercial applications may not be available.

7. Conclusion

Open-source Operating Systems allow users to study, modify and share software according to their licences. They provide flexibility, customization and community support.

Remember: Open source means the source code is available to study and modify under its licence.

Q7. Explain Layered and Microkernel Structures of Operating Systems with neat sketch.

5 Marks – 10 Marks Format

1. Introduction

Operating System structures describe how the different parts of an OS are arranged.

Two important structures are:

1. Layered Structure.
2. Microkernel Structure.

Both structures organize OS components, but they work differently.

2. Layered Structure

A Layered Operating System is divided into different levels called layers.

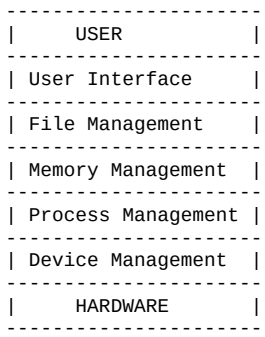
Each layer performs a specific task. One layer uses the services of the layer below it.

Working:

1. The user gives a command.
2. The command reaches the upper layer.
3. Each layer performs its work.
4. The request moves through the required lower layers.
5. Finally, the hardware performs the operation.

Example: In a layered OS, a file request passes through different OS layers before reaching the storage device.

3. Diagram – Layered Structure



4. Advantages of Layered Structure

1. Easy to understand.
2. Easy to maintain.
3. Each layer has a separate responsibility.
4. Errors can be identified easily.
5. Changes in one layer may have limited effects on others.

5. Disadvantages of Layered Structure

1. Designing proper layers is difficult.
2. Communication between layers can take extra time.
3. Too many layers may reduce performance.

6. Microkernel Structure

A Microkernel Operating System keeps only the most essential services inside the kernel.

Other services work outside the kernel as separate user-space processes.

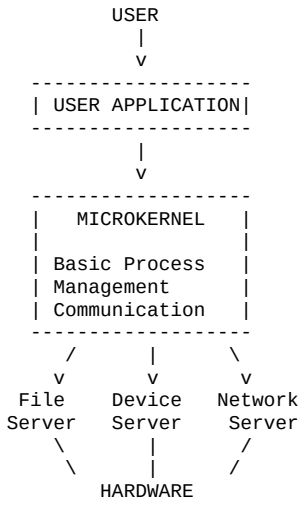
The essential kernel functions usually include basic process management, memory management and inter-process communication.

Working:

1. The user gives a request.
2. The request reaches the microkernel.
3. The microkernel communicates with the required service.
4. The service performs the operation.
5. The result is returned to the user.

Example: MINIX 3 uses a microkernel-based design.

7. Diagram – Microkernel Structure



8. Advantages of Microkernel

1. Small kernel size.
2. Better isolation between services.
3. Easier to maintain individual services.
4. A failure in one service may not crash the entire OS.
5. More secure design when services are properly isolated.

9. Disadvantages of Microkernel

1. Communication between services may take extra time.
2. More complex communication is required.
3. Performance may be affected by frequent message passing.

10. Difference Between Layered and Microkernel Structures

Layered Structure	Microkernel Structure
OS is divided into layers.	OS is divided into essential kernel and external services.
Each layer performs a specific task.	Only essential services run inside the kernel.
Layers communicate with nearby layers according to the design.	Services communicate through message passing.
Focuses on organized levels.	Focuses on keeping the kernel small.
Example: THE operating system.	Example: MINIX 3.

11. Conclusion

Layered structure organizes an OS into different levels, while microkernel structure keeps only essential functions inside the kernel. Both structures help organize OS components and improve maintainability.

Remember:

- Layered = Different levels.
- Microkernel = Small kernel + external services.

Q8. Define an Operating System and explain its main objectives.

5 Marks – 10 Marks Format

1. Definition

An Operating System is system software that acts as an interface between the user and computer hardware.

It manages computer resources and helps users run application programs.

Examples: Windows, Linux, Android and macOS.

2. Main Objectives of an Operating System

1. Convenience

- The OS makes the computer easy to use.
- It provides an interface to users.
- Users can perform tasks without directly controlling hardware.

Example: We can open a file by clicking its icon.

2. Efficiency

- The OS uses computer resources properly.
- It manages CPU, memory and storage.
- It avoids unnecessary resource wastage.

Example: The OS shares CPU time among running applications.

3. Resource Management

- The OS manages all computer resources.
- It allocates CPU time and memory to programs.
- It manages input and output devices.

Example: Giving RAM to a browser when it is opened.

4. Security and Protection

- The OS protects user information.
- It provides passwords and access control.
- It prevents unauthorized access.

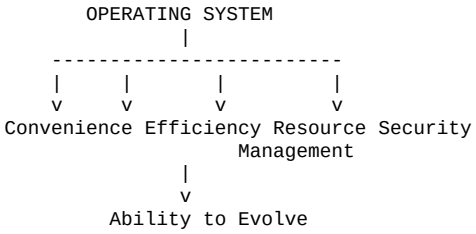
Example: A password protects a computer account.

5. Ability to Evolve

- The OS should support new features.
- It should allow updates and improvements.
- It should support new hardware and technologies.

Example: Updating Windows to get new features and security fixes.

3. Diagram – Objectives of an Operating System



4. Other Important Objectives

1. Reliability – The computer should work properly.
2. Performance – The OS should provide good performance.
3. User Interaction – It should help users communicate with the computer.
4. Error Handling – It should detect and handle errors.

5. Real-Life Example

Think of an Operating System as a school principal.

- The principal manages teachers and students.
- The principal distributes responsibilities.
- The principal maintains discipline.
- The principal ensures the school works properly.

Similarly, an OS manages all computer activities.

6. Conclusion

An Operating System is essential for a computer. Its main objectives are to make the computer easy to use, manage resources efficiently, provide security and support future improvements.

Remember: The main objectives are Convenience, Efficiency and Ability to Evolve.

Q9. Explain the steps involved in building and booting an Operating System.

5 Marks – 10 Marks Format

1. Introduction

Building an Operating System means preparing the OS software to run on a computer.

Booting is the process of starting a computer and loading the Operating System into memory.

There are two important parts:

1. Building an OS.
2. Booting an OS.

2. Steps Involved in Building an Operating System

1. Selecting the OS Source Code

- Developers select the OS source code.
- They decide which features are required.
- They select the necessary OS components.

Example: Selecting Linux source code.

2. Configuring the OS

- Configuration means selecting the required settings.
- Developers choose hardware support.

- They select required features and drivers.

Example: Configuring an OS for a laptop.

3. Compiling the Source Code

- Compilation converts source code into machine-understandable instructions.
- A compiler is used for this process.
- It prepares the OS components for execution.

Example: Compiling the Linux kernel.

4. Linking the Components

- Linking combines the required compiled components.
- It connects different program parts.
- It helps prepare the final OS files.

5. Installing the OS

- The prepared OS is installed on a storage device.
- Required system files are copied.
- Boot-related files are also configured.

Example: Installing Ubuntu on a computer.

3. Steps Involved in Booting an Operating System

1. Power On

When we press the power button, the computer starts receiving power.

2. BIOS/UEFI Starts

BIOS or UEFI is firmware that starts the computer hardware and begins the boot process.

3. POST

POST means Power-On Self-Test.

It checks whether important hardware components are working properly.

4. Bootloader Loads

The bootloader is a small program that helps load the Operating System.

5. Kernel Loads

The bootloader loads the OS kernel into RAM.

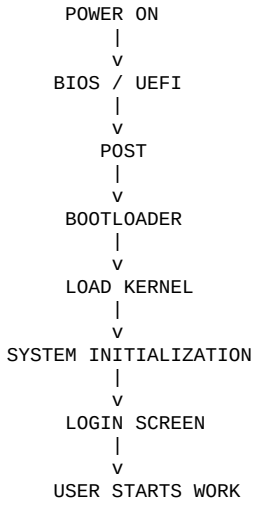
6. System Initialization

The kernel initializes hardware, memory and essential system services.

7. Login Screen Appears

Finally, the login screen or desktop appears, and the user can start working.

4. Diagram – Booting Process



5. Types of Booting

1. Cold Booting

- Starting a computer from a completely powered-off state.
- Example: Switching on a computer in the morning.

2. Warm Booting

- Restarting a computer without completely switching off its power.
- Example: Selecting Restart from the Start menu.

6. Conclusion

Building an OS involves selecting, configuring, compiling and installing its components. Booting starts the computer and loads the kernel into memory so that users can work.

Remember: Power On → BIOS/UEFI → POST → Bootloader → Kernel → Login.

Q10. What are the main services provided by an Operating System?

5 Marks – 10 Marks Format

1. Introduction

Operating System services are the facilities provided by the OS to users and application programs.

These services make the computer easy to use and help programs work properly.

2. Main Services of an Operating System

1. User Interface

- It provides a way for users to communicate with the computer.
- It accepts commands and user actions.
- It can be CLI, GUI or touch interface.

Example: Using the Windows desktop.

2. Program Execution

- The OS loads programs into memory.
- It starts their execution.
- It manages the program while it is running.
- It handles program termination.

Example: Opening and running a calculator.

3. I/O Operations

- I/O means Input and Output.
- The OS manages input and output devices.
- It allows programs to communicate with devices.
- It controls devices using drivers.

Example: Typing through a keyboard and displaying text on the monitor.

4. File System Management

- The OS manages files and folders.
- It allows users to create and delete files.
- It supports reading and writing data.
- It controls file permissions.

Example: Saving a photograph in a folder.

5. Communication

- The OS allows processes to exchange information.
- It supports communication between programs.
- It can also support communication over networks.

Example: Two applications exchanging data.

6. Error Detection

- The OS checks for errors.
- It detects hardware and software problems.
- It takes suitable action to handle errors.

Example: Displaying an error when a storage device is not available.

7. Resource Allocation

- The OS distributes computer resources.
- It allocates CPU time, memory and devices.
- It manages resources among multiple programs.

Example: Sharing CPU time between Chrome and a music player.

8. Accounting

- The OS keeps track of resource usage.
- It monitors CPU time and memory usage.
- It helps administrators understand system usage.

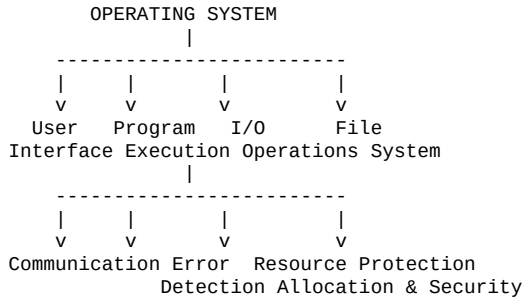
Example: Tracking how much CPU time a process uses.

9. Protection and Security

- The OS protects data and resources.
- It checks user permissions.
- It prevents unauthorized access.

Example: Asking for a password before opening a user account.

3. Diagram – Operating System Services



4. Advantages of OS Services

1. Makes the computer easy to use.
2. Helps programs execute properly.
3. Manages hardware resources.
4. Protects user data.
5. Improves system reliability.

5. Conclusion

Operating System services provide important facilities such as program execution, file management, I/O operations, communication, error detection, resource allocation and security.

Remember: OS services help users and application programs use computer resources easily and safely.

Q11. Explain about the Dual Mode Operation in OS with a neat block diagram.

5 Marks – 10 Marks Format

1. Introduction

Dual Mode Operation is a security feature of an Operating System.

It divides computer operation into two modes:

1. User Mode.
2. Kernel Mode.

These modes help protect the computer from incorrect or harmful operations.

Example: A normal application runs in User Mode, while the OS performs important hardware operations in Kernel Mode.

2. Types of Modes

1. User Mode

- User Mode is used to run application programs.
- Programs have limited access to hardware.
- They cannot directly execute privileged instructions.
- They must request OS services through system calls.

Example: Running Chrome, MS Word or a calculator.

2. Kernel Mode

- Kernel Mode is also called Supervisor Mode.
- The OS kernel runs in this mode.
- It has access to hardware and system resources.
- It can execute privileged instructions.

• It manages memory, CPU and devices.

Example: The OS controlling the printer or managing RAM.

3. Mode Bit

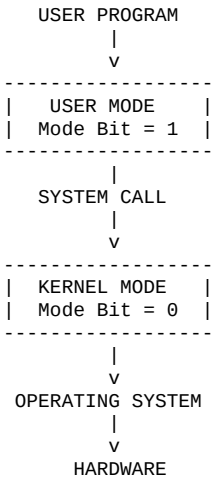
A mode bit is a special bit used to identify the current mode of operation.

• Mode bit = 1 → User Mode.

• Mode bit = 0 → Kernel Mode.

Note: This is a commonly used textbook convention; actual hardware conventions can differ.

4. Diagram – Dual Mode Operation



5. Working of Dual Mode Operation

1. The computer starts an application in User Mode.
2. The application performs normal operations.
3. When it needs an OS service, it makes a system call.
4. The CPU switches to Kernel Mode.
5. The OS performs the requested operation.
6. After completing the operation, the CPU returns to User Mode.

Example: When an application wants to read a file, it requests the OS through a system call. The OS accesses the file and returns the data.

6. Advantages of Dual Mode Operation

1. Protects the Operating System.
2. Prevents applications from directly accessing protected hardware.
3. Improves system security.
4. Prevents unauthorized operations.
5. Helps maintain system stability.

7. Conclusion

Dual Mode Operation protects the computer by dividing its working into User Mode and Kernel Mode. It allows applications to work safely while giving the OS control over important hardware operations.

Remember: User Mode = Limited Access; Kernel Mode = Full Privileged Access.

Q12. What is an Operating System? Explain Computing Environments.

5 Marks – 10 Marks Format

1. Definition of Operating System

An Operating System is system software that acts as an interface between the user and computer hardware.

It manages computer resources and allows application programs to run.

Examples: Windows, Linux, Android and macOS.

2. Computing Environment

A Computing Environment describes the type of computer system and the way users and programs use its resources.

Different computing environments are used for different purposes.

3. Types of Computing Environments

1. Traditional Computing

- Traditional computing uses personal computers and workstations.
- Users perform tasks on their own computers.
- It may be connected to a network.
- It is commonly used in homes, schools and offices.

Example: Using a desktop computer to prepare documents.

2. Mobile Computing

- Mobile computing uses portable devices.
- Users can work while moving from one place to another.
- It uses wireless networks and mobile applications.
- It provides communication and internet access.

Example: Using an Android smartphone or tablet.

3. Client-Server Computing

- In this environment, computers communicate using a network.
- A client requests a service.
- A server provides the requested service.
- The server manages shared resources.

Example: A web browser requests a webpage from a web server.

4. Peer-to-Peer Computing

- Peer-to-Peer is also called P2P computing.
- Computers communicate directly with each other.
- Each computer can act as both a client and a server.
- Resources can be shared between computers.

Example: Sharing files directly between computers using a P2P application.

5. Cloud Computing

- Cloud computing provides computing services through the internet.
- Users can access storage, software and processing power online.
- They do not need to maintain all resources on their own computers.
- Services can be accessed from different locations.

Example: Google Drive and Microsoft OneDrive.

6. Real-Time Embedded Computing

- Embedded systems are designed for specific tasks.

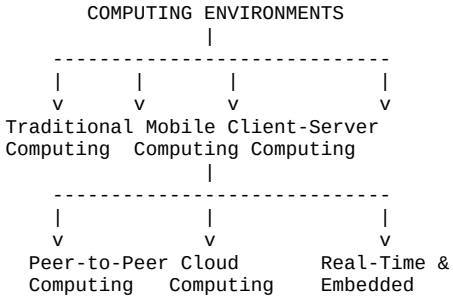
• Real-time systems must respond within a required time limit.

• They are used in machines and electronic devices.

• They perform dedicated operations.

Example: An airbag system in a car must respond quickly during an accident.

4. Diagram – Computing Environments



5. Advantages of Computing Environments

1. Support different types of devices.
2. Allow resource sharing.
3. Improve communication.
4. Provide online storage and services.
5. Support specific tasks in different industries.

6. Conclusion

An Operating System manages computer hardware and software. Computing environments describe how computer systems are used, such as traditional, mobile, client-server, peer-to-peer, cloud and real-time computing.

Remember: Computing environments explain where and how computing resources are used.

Q13. Write the differences between Monolithic Kernel and Microkernel.

5 Marks – 10 Marks Format

1. Introduction

A Kernel is the main part of an Operating System. It manages important resources like CPU, memory and devices.

There are different types of kernel designs.

Two important types are:

1. Monolithic Kernel.
2. Microkernel.

2. Monolithic Kernel

A Monolithic Kernel is an OS design in which most Operating System services run inside one large kernel.

These services include:

• Process management.

• Memory management.

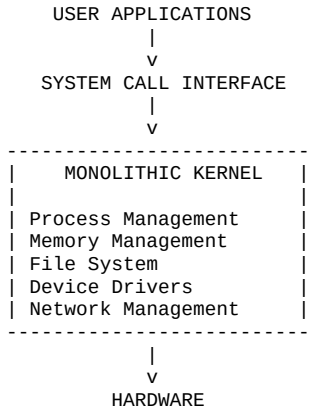
• File management.

• Device drivers.

• Network management.

Example: Linux uses a monolithic kernel with a modular design.

3. Diagram – Monolithic Kernel



4. Advantages of Monolithic Kernel

1. High performance in many operations.
2. Services communicate directly inside the kernel.
3. Fast communication between components.
4. Efficient resource management.

5. Disadvantages of Monolithic Kernel

1. Large kernel size.
2. A serious bug in kernel code can crash the entire system.
3. Difficult to maintain in complex systems.
4. A large amount of code runs with high privileges.

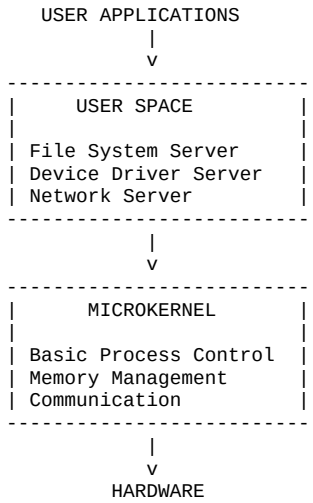
6. Microkernel

A Microkernel is an OS design in which only essential services run inside the kernel.

Other services, such as file systems and device drivers, run outside the kernel as separate user-space processes.

Example: MINIX 3 uses a microkernel architecture.

7. Diagram – Microkernel



8. Advantages of Microkernel

1. Small kernel size.
2. Better isolation between services.
3. Easier to maintain separate services.

4. A failure in one external service may not crash the whole OS.
5. Provides a smaller trusted kernel.

9. Disadvantages of Microkernel

1. Communication between services may take extra time.
2. Message passing creates overhead.
3. Designing the system can be complex.

10. Differences Between Monolithic Kernel and Microkernel

Monolithic Kernel	Microkernel
Large kernel.	Small kernel.
Most OS services run inside the kernel.	Only essential services run inside the kernel.
Less communication overhead between internal services.	Uses message passing between services.
A kernel bug can affect the entire system.	External service failures can be isolated.
Generally offers efficient internal communication.	Communication overhead may reduce performance.
Example: Linux.	Example: MINIX 3.

11. Real-Life Example

Imagine a school:

- **Monolithic:** All teachers, office staff and other departments work inside one large management office.
- **Microkernel:** Only the principal and essential staff are in the main office. Other departments work separately and communicate with the main office.

12. Conclusion

A Monolithic Kernel keeps most OS services inside one large kernel, while a Microkernel keeps only essential services inside the kernel. Both designs have advantages and disadvantages.

Remember:

- Monolithic = Most services inside the kernel.
- Microkernel = Essential services inside the kernel.

Q14. Explain briefly User and Operating System Interface (CLI, GUI, Touch).

5 Marks – 10 Marks Format

1. Introduction

A User Interface is a way through which a user communicates with a computer.

It helps users give instructions to the Operating System and receive results.

There are three important types:

1. CLI – Command Line Interface.
2. GUI – Graphical User Interface.
3. Touch Interface.

2. Command Line Interface (CLI)

- CLI allows users to communicate with the computer by typing commands.
- Users must enter instructions using a keyboard.
- The computer executes the given commands.
- It is useful for system administrators and programmers.

Example: Using Command Prompt in Windows.

Common commands:

```
dir
```

Displays files and folders in Windows Command Prompt.

```
mkdir test
```

Creates a folder named test.

3. Diagram – CLI

```
     USER
       |
       v
   TYPES COMMAND
       |
       v
      CLI
       |
       v
  OPERATING SYSTEM
       |
       v
    RESULT
```

4. Graphical User Interface (GUI)

- GUI allows users to communicate through graphical elements.
- It uses icons, windows, menus and buttons.
- Users can perform tasks by clicking with a mouse.
- It is easy for beginners to understand.

Example: Windows desktop.

5. Diagram – GUI

```
    -------------------------
    | ICONS                 |
    |  [Folder] [Browser]   |
    |                       |
    |      WINDOWS          |
    |                       |
    |-----------------------|
    | START MENU | TASKBAR  |
    -------------------------
```

6. Touch Interface

- Touch Interface allows users to operate devices by touching the screen.
- Users can tap, swipe, drag and pinch.
- It is mainly used in smartphones and tablets.
- It makes device operation simple and direct.

Example: Using an Android mobile phone.

7. Diagram – Touch Interface

```
        TOUCH SCREEN
       ---------------
       |  [ APP 1 ]  |
       |             |
       |  [ APP 2 ]  |
       |             |
       |  [ APP 3 ]  |
       |             |
       ---------------
              ^
              |
        FINGER TOUCH
```

8. Differences Between CLI, GUI and Touch Interface

CLI	GUI	Touch
Uses typed commands.	Uses graphical elements.	Uses finger or stylus.
Requires command knowledge.	Easy to learn.	Easy to operate.
Mainly uses keyboard.	Uses mouse and keyboard.	Uses touchscreen.
Suitable for advanced users.	Suitable for general users.	Suitable for mobile devices.
Example: Command Prompt.	Example: Windows desktop.	Example: Android phone.

9. Advantages

CLI:

1. Fast for experienced users.
2. Uses fewer graphical resources.
3. Useful for automation.

GUI:

1. Easy to understand.
2. Uses icons and menus.
3. Suitable for beginners.

Touch Interface:

1. Easy to operate.
2. Supports direct interaction.
3. Useful for smartphones and tablets.

10. Conclusion

User Interfaces help people communicate with an Operating System. CLI uses commands, GUI uses graphical elements and Touch Interface uses screen touches.

Remember:

• CLI = Type.

• GUI = Click.

• Touch = Tap and Swipe.

Q15. Write short notes on Operating System Design and Implementation.

5 Marks – 10 Marks Format

1. Introduction

Operating System Design and Implementation explains how an Operating System is planned, developed and created.

• **Design** means planning how the OS should work.

• **Implementation** means converting that design into a working Operating System using programming languages.

Example: Before building a house, we prepare a plan. Similarly, developers design an OS before implementing it.

2. Operating System Design

Operating System Design is the process of deciding the structure, features and working of an OS.

1. User Goals

• The OS should be easy to use.

- It should be simple and convenient.
- It should provide a good user experience.

Example: Users should easily open files and applications.

2. System Goals

- The OS should be efficient.
- It should provide good performance.
- It should be reliable and secure.
- It should be easy to maintain.

Example: The OS should manage memory properly without crashing.

3. Selection of OS Structure

- Developers select a suitable OS structure.
- They decide how different components work together.
- Common structures include monolithic, layered and microkernel.

Example: Choosing a layered structure for organizing OS components.

3. Operating System Implementation

Implementation means developing the Operating System using programming languages.

1. Programming Languages

- OS developers use programming languages to write OS code.
- C and C++ are commonly used.
- Assembly language is also used for low-level operations.

Example: The Linux kernel is mainly written in C, with some assembly language.

2. Writing the Code

- Developers write code for different OS components.
- They develop process and memory management.
- They also develop file systems and device drivers.

3. Testing

- The OS is tested after development.
- Developers check whether all components work properly.
- They identify and correct errors.

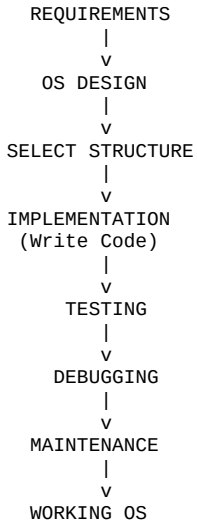
4. Debugging

- Debugging means finding and fixing errors.
- It improves system performance.
- It makes the OS more reliable.

5. Maintenance

- Developers release updates.
- They fix security problems.
- They improve existing features.

4. Diagram – OS Design and Implementation



5. Important Principles

1. **Modularity:** Divide the OS into smaller components.
2. **Security:** Protect data and system resources.
3. **Reliability:** Ensure the OS works properly.
4. **Efficiency:** Use resources properly.
5. **Maintainability:** Make the OS easy to update and modify.

6. Advantages

1. Helps create an organized OS.
2. Improves system performance.
3. Makes testing easier.
4. Reduces errors.
5. Makes future updates easier.

7. Conclusion

Operating System Design and Implementation is an important process in OS development. Design decides how the OS should work, while implementation converts the design into a working system.

Remember: Design = Planning; Implementation = Developing the OS.

Q16. Explain System Programs in detail.

5 Marks – 10 Marks Format

1. Introduction

System Programs are software tools that provide a convenient environment for users to work with a computer.

They help users perform common tasks such as managing files, developing programs and checking system information.

They work with the services provided by the Operating System.

Examples: File managers, command interpreters, compilers and system utilities.

2. Types of System Programs

1. File Management

- File management programs help users manage files and folders.
- They allow users to create, delete, copy and rename files.

- They help organize data.

Example: Windows File Explorer.

2. Status Information

- These programs provide information about the computer.
- They display system details.
- They show memory, storage and CPU usage.
- They provide information about running processes.

Example: Windows Task Manager.

3. File Modification

- These programs help users modify the contents of files.
- They allow users to edit and update information.
- They can search and replace text.

Example: Notepad and text editors.

4. Programming Language Support

- These programs help programmers develop software.
- They provide compilers, assemblers and interpreters.
- They convert program instructions into executable form or execute them.

Examples:

- GCC – C compiler.
- Python Interpreter – Executes Python programs.

5. Program Loading and Execution

- These programs help load programs into memory.
- They prepare programs for execution.
- They start the program and support its execution.

Example: A program loader loads an application into memory.

6. Communication Programs

- These programs help computers communicate with each other.
- They support sending and receiving information.
- They help users share data through networks.

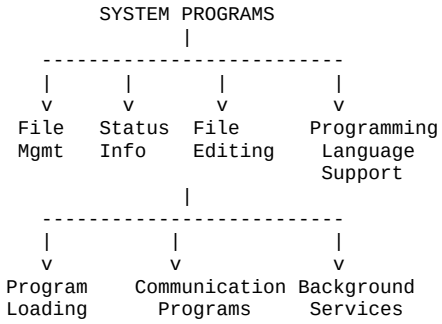
Example: SSH and network communication utilities.

7. Background Services

- These programs run in the background.
- They perform important system tasks.
- They may start automatically when the computer starts.

Example: A printing service manages print jobs.

3. Diagram – Types of System Programs



4. Real-Life Example

Imagine you are using a computer to prepare a college assignment.

1. You open File Explorer to find a document.
2. You use Notepad to edit text.
3. You use a compiler to run a program.
4. You use Task Manager to check memory usage.
5. You use a printing service to print your assignment.

All these tools are examples of system programs.

5. Advantages of System Programs

1. Make computer usage easier.
2. Help manage files and folders.
3. Support software development.
4. Provide system information.
5. Help programs load and execute.
6. Support communication between computers.

6. Conclusion

System Programs provide a convenient environment for users and programmers. They support file management, program development, system monitoring, communication and program execution.

Remember: System Programs provide useful tools to work with the Operating System.